

→JS-3

→String Methods

String Methods

Methods - actions that can be performed on objects.

Format

stringName.**method()**

let str = "apna college"

str.abcd()

→Trim Method

String Methods

```
let msg = "  Hello  ";
```

str.trim()

Trims whitespaces from both ends of string & returns a new one.

```
> let msg = "  Hello  ";
< undefined
> msg.trim();
< 'Hello'
> msg
< '  Hello  '
```

output : "Hello", but value of msg remains same.

→ Strings are Immutable in js

String are Immutable in JS

No changes can be made to strings.

Whenever we do try to make a change, a new string is created and old one remains same.

```
top Filter
> let msg = "apna ";
< undefined
> msg.trim();
< 'apna'
> let str = msg.trim();
< undefined
> str
< 'apna'
> msg
< 'apna'
> msg = "hello";
< 'hello'
> msg
< 'hello'
>
```

→ ToUppercase and ToLowercase

String Methods

```
let str = "Random string";
```

```
str.toUpperCase() "RANDOM STRING"
```

```
str.toLowerCase() "random string"
```

```
top Filter
> name
< 'Apna College'
> name.toLowerCase();
< 'apna college'
> name.toLowerCase
< f toLowerCase() { [native code] }
> name.toUpperCase();
< 'APNA COLLEGE'
```

→String methods with Arguments

String Methods with Arguments

Argument is a some value that we pass to the method.

Format

stringName.method(arg)

→Index Of

indexOf

Returns the **first index of occurrence** of some value in string. Or gives -1 if not found.

let str = "IloveCoding";
0 1 2 3 4

str.indexOf("love") // 1 ✓

str.indexOf("J") // -1 (not found)

str.indexOf("o") // 2 (only 1 index)

```
> msg
< 'ILoveCoding'
> msg.indexOf("Love");
< 1
> msg.indexOf("love");
< -1
> msg.indexOf("o");
< 2
> msg.indexOf("z");
< -1
```

→Method Chaining

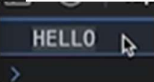
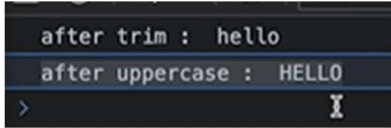
Method Chaining

Using one method after another. Order of execution will be left to right.

str.toUpperCase().trim()

```
let msg = "  hello  ";
let newMsg = msg.trim();
console.log("after trim : ", newMsg);
newMsg = newMsg.toUpperCase();
console.log("after uppercase : ", newMsg);

// console.log("after uppercase : ", newMsg);
let newMsg = msg.trim().toUpperCase();
```



→Slice

slice

Returns a part of the original string as a new string.

let str = "IloveCoding";

str.slice(5) // "Coding"

str.slice(1, 4) // "love"

str.slice(-num) = str.slice(length-num)

slice

Returns a part of the original string as a new string.

↓
let str = "IloveCoding";
 0 1
str.slice(5) // "Coding" creddy2096@gmail.com

str.slice(1, 4) // "love"

str.slice(-num) = str.slice(length-num)

str.slice(start, end) APNA COLLEGE
 1, 5

str.slice(start)
 end = str.length

str.slice(-ve)
 -2
 [str.length - 2]



→ Replace and Repeat

replace

Searches a value in the string & returns a new string with the value replaced.

```
let str = "IloveCoding";  
  
str.replace("love", "do") // "IdoCoding"  
  
str.replace("o", "x")     // "IlxveCoding"
```

```
> msg  
< 'IloveCoding'  
> msg.replace("love", "do");  
< 'IdoCoding'  
> msg  
< 'IloveCoding'  
> msg.replace('o', 'x');  
< 'IlxveCoding'  
> msg = "apnaapnaapna";  
< 'apnaapnaapna'  
> msg.replace("apna", "college");  
< 'collegeapnaapna'
```

→ Repeat

repeat

Returns a string with the number of copies of a string

```
let str = "Mango";
```

```
str.repeat(3) // "MangoMangoMango"
```

→ Array(Data Structure)

Array (Data Structure)

Linear collection of things



```
app.js > ...  
// let student1 = "aman";  
// let student2 = "shradha";  
// let student3 = "rajat";  
  
let students = ["aman", "shradha", "rajat"];
```

```
> students  
< ▾ (3) ['aman', 'shradha', 'rajat']  
  0: "aman"  
  1: "shradha"  
  2: "rajat"  
  length: 3  
  ► [[Prototype]]: Array(0)
```

→ Visualizing Array

2	4	6	8
0	1	2	3

let nums = [2, 4, 6, 8];

```
> let nums = [2, 4, 6, 8];
< undefined
> nums
< ▶ (4) [2, 4, 6, 8]
> let name = "shradha";
< undefined
> name[0]
< 's'
> nums[0]
< 2
> nums[1]
< 4
> nums[2]
< 6
> nums[3]
< 8
> nums[4]
< undefined
> nums.length
< 4

> let marks = [99, 89, 67, 42, 100];
< undefined
> marks
< ▶ (5) [99, 89, 67, 42, 100]
> students[0]
< 'aman'
> students[1]
< 'shradha'
>
```

→ Creating arrays

Creating Arrays

```
let marks = [99, 85, 93, 76, 62];
let names = ["adam", "bob", "catlyn"];
let info = ["aman", 25, 6.1]; //mixed array

//empty array
let newArr = [];
```

arr.length

```
> let empArr = [];
< undefined
> empArr
< ▼ []
  length: 0
  ▶ [[Prototype]]: Array(0)

> info.length
< 3
> empArr.length
< 0
> | x
```

→ to access the character of the arrays element we a[0][0].

```
> info
< ▶ (3) ['shradha', 23, 89.9]
> let empArr = [];
< undefined
> empArr
< ▶ []
> empArr[0]
< undefined
> info.length
< 3
> empArr.length
< 0
> [].length
< 0
> [1, 2, 3, 4].length
< 4
> info[0]
< 'shradha'
> info[0][0]
< 's'
> info[0][1]
< 'h'
> info[0].length
< 7
```

→ Arrays are Mutable

Arrays are Mutable

```
> let fruits = ["mango", "apple", "litchi"];
< undefined
> fruits[0] = "banana";
< 'banana'
> fruits
< ▶ (3) ['banana', 'apple', 'litchi']
```

```

> let name = "rohit";
< undefined
> name[0] = 'm';
< 'm'
> name
< 'rohit'
> let fruits = ["mango", "apple", "litchi"];
< undefined
> fruits
< ▶ (3) ['mango', 'apple', 'litchi']
> fruits[0] = "banana";
< 'banana'
> fruits
< ▶ (3) ['banana', 'apple', 'litchi']
> fruits[1] = "pineapple";
< 'pineapple'
> fruits
< ▶ (3) ['banana', 'pineapple', 'litchi']
> fruits[10] = "papaya";
< 'papaya'
> fruits
< ▶ (11) ['banana', 'pineapple', 'litchi', empty × 7, 'papaya']
>
fruits
▼ (11) ['banana', 'pineapple', 'litchi', empty × 7, 'papaya'] ⓘ
  0: "banana"
  1: "pineapple"
  2: "litchi"
  10: "papaya"
  length: 11
  ▶ [[Prototype]]: Array(0)

```

→ Array Methods

Push : add to end

Unshift : add to start

Pop : delete from end & returns it

Shift : delete from start & returns it

→ shift and unshift

```
< > (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.unshift("toyota");
< 5
> cars
< > (5) ['toyota', 'audi', 'bmw', 'xuv', 'maruti']
> cars.unshift("ferrari");
< 6
> cars
< > (6) ['ferrari', 'toyota', 'audi', 'bmw', 'xuv', 'maruti']
> cars.shift();
< 'ferrari'
> cars
< > (5) ['toyota', 'audi', 'bmw', 'xuv', 'maruti']
> let followers = ["a", "b", "c"];
< undefined
> let blocked = followers.shift();
< undefined
> followers
< > (2) ['b', 'c']
> blocked
< 'a'
```

→ push and pop

```
top Filter Defa
> cars
< > (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.push("toyota");
< 5
> cars
< > (5) ['audi', 'bmw', 'xuv', 'maruti', 'toyota']
> cars.push("ferrari");
< 6
> cars
< > (6) ['audi', 'bmw', 'xuv', 'maruti', 'toyota', 'ferrari']
> cars.pop();
< 'ferrari'
> cars
< > (5) ['audi', 'bmw', 'xuv', 'maruti', 'toyota']
> cars.pop();
< 'toyota'
> cars
< > (4) ['audi', 'bmw', 'xuv', 'maruti']
>
```

→ Array methods

→ indexOf: return index of something

Array Methods

indexOf : returns index of something

```
> let primary = ["red", "yellow", "blue"];
                    ↑       ↑       ↑
                    0       1       2
```

```
> primary.indexOf("yellow");
< 1
> primary.indexOf("green");
< -1
> primary.indexOf("Yellow");
< -1
```

includes : search for a value

```
> primary.includes("red");
< true
> primary.includes("green");
< false
```



Array Methods

indexOf : returns index of something

```
> let primary = ["red", "yellow", "blue"];
                    ↑       ↑       ↑
                    0       1       2
```

found → index
not found → -1

```
> primary.indexOf("yellow");
< 1 ✓
> primary.indexOf("green");
< -1
> primary.indexOf("Yellow");
< -1
```

includes : search for a value

found → true
not found → false

```
> primary.includes("red");
< true
> primary.includes("green");
< false
```

jcreddy2096@gmail.com



→ IndexOf:

```

> cars
< ▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.indexOf("bmw");
< 1
> cars.indexOf("xuv");
< 2
> cars.indexOf("XUV");
< -1
> let marks = [99, 89, 67, 42, 100];
< undefined
> marks.indexOf(100);
< 4
> marks.indexOf(97);
< -1

```

→concat:

Array Methods

concat : merge 2 arrays → *concatenate*

```

> primary.concat(secondary);
< ▶ (6) ['red', 'yellow', 'blue', 'orange', 'green', 'violet']

```

reverse : reverse an array

```

> primary.reverse();
< ▶ (3) ['blue', 'yellow', 'red']

```

jcreddy2096@gmail.com

```

> let primary = ["red", "yellow", "blue"];
< undefined
> let secondary = ["orange", "green", "violet"];
< undefined

```

→in reverse the original method only will changed we don't get the new array like the concat

→Slice

Array Methods

slice (start, end)
optional

slice : copies a portion of an array

```
> let colors = ["red", "yellow", "blue", "orange", "pink", "white"];  
  
> colors.slice()  
< ▶ (6) ['red', 'yellow', 'blue', 'orange', 'pink', 'white']  
  
> colors.slice(2);  
< ▶ (4) ['blue', 'orange', 'pink', 'white']  
  
> colors.slice(2, 3);  
< ▶ ['blue']  
  
> colors.slice(-2);  
< ▶ (2) ['pink', 'white']
```



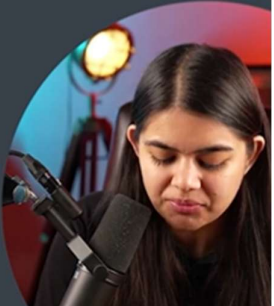
→Splice in arrays

Array Methods

splice : removes / replaces / add elements in place

splice(start, deleteCount, item0...itemN)

```
> let colors = ["red", "yellow", "blue", "orange", "pink", "white"]  
  
> colors.splice(4);  
< ▶ (2) ['pink', 'white']  
  
> colors  
< ▶ (4) ['red', 'yellow', 'blue', 'orange']  
  
> colors.splice(0, 1);  
< ▶ ['red']  
  
> colors  
< ▶ (3) ['yellow', 'blue', 'orange']  
  
> colors.splice(0, 1, "black", "grey");  
< ▶ ['yellow']  
  
> colors  
< ▶ (4) ['black', 'grey', 'blue', 'orange']
```



→Difference between the slice and splice is slice creates an copy of original splice makes changes in the original itself.

→ Sort in Arrays

Array Methods

sort : sorts an array

```
> let days = ["monday", "sunday", "wednesday", "tuesday"];  
< undefined  
> days.sort();  
< ▶ (4) ['monday', 'sunday', 'tuesday', 'wednesday']
```

```
> let squares = [25, 16, 4, 49, 36, 9]  
< undefined  
> squares.sort();  
< ▶ (6) [16, 25, 36, 4, 49, 9]
```

```
> cars  
< ▶ (6) ['toyota', 'gwagon', 'xuv', 'maruti', 'bmw', 'ferrari']  
> cars.sort();  
< ▶ (6) ['bmw', 'ferrari', 'gwagon', 'maruti', 'toyota', 'xuv']  
> let chars = ['b', 'd', 'e', 'a'];  
< undefined  
> chars.sort();  
< ▶ (4) ['a', 'b', 'd', 'e']  
> let marks = [99, 89, 67, 42, 100];  
< undefined  
> marks.sort();  
< ▶ (5) [100, 42, 67, 89, 99]
```

→ Arrays References

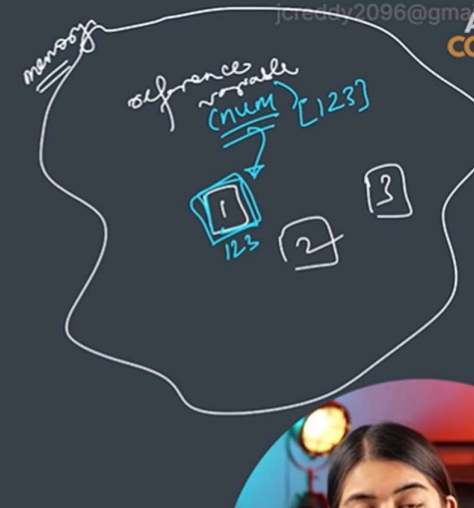
Array References

```
> [1] === [1]
< false
> [1] == [1]
< false
```

address in memory

let num = [1, 2, 3]

ref num



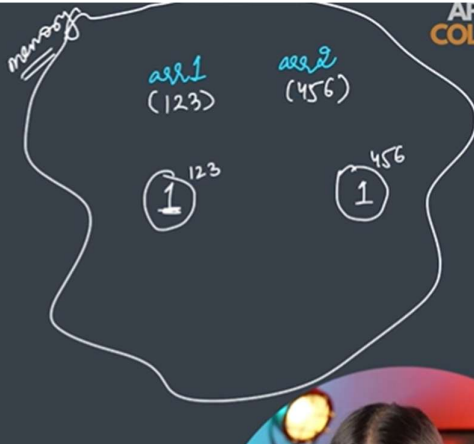
Array References

```
> [1] === [1]
< false
> [1] == [1]
< false
```

address in memory

let arr1 = [1] ✓

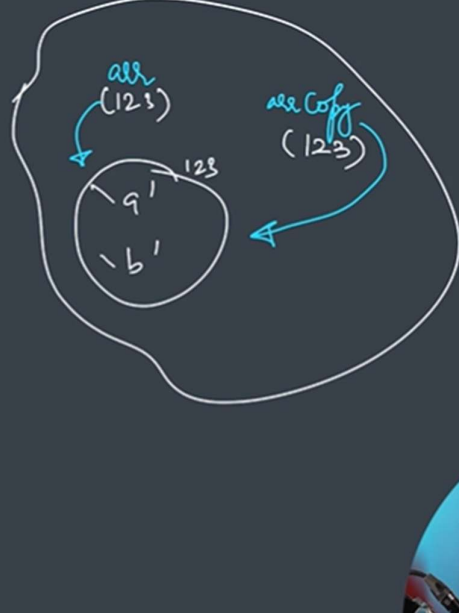
let arr2 = [1] ✓



Array References

```
> let arr = ['a', 'b'];
< undefined
> let arrCopy = arr;
< undefined
> arrCopy
< ▶ (2) ['a', 'b']
> arrCopy.push('c');
< 3
> arr
< ▶ (3) ['a', 'b', 'c']

> arr == arrCopy
< true
```



```

> let arr = ['a', 'b', 'c'];
< undefined
> let arrCopy = arr;
< undefined
> arr == arrCopy
< true
> arr === arrCopy
< true
> arr.push('d');
< 4
> arr
< ▶ (4) ['a', 'b', 'c', 'd']
> arrCopy
< ▶ (4) ['a', 'b', 'c', 'd']
> arrCopy.pop();
< 'd'
> arrCopy
< ▶ (3) ['a', 'b', 'c']
> arr
< ▶ (3) ['a', 'b', 'c']
>

```

→ if you create an new array then it will change

```

> arr
< ▶ (3) ['a', 'b', 'c']
> arrCopy = ['a', 'b', 'c'];
ⓧ Uncaught SyntaxError: Invalid
> arrCopy = ['a', 'b', 'c'];
< ▶ (3) ['a', 'b', 'c']
> arr.push('d');
< 4
> arr
< ▶ (4) ['a', 'b', 'c', 'd']
> arrCopy
< ▶ (3) ['a', 'b', 'c']
>

```

→ Constant Arrays

Constant Arrays

```
const arr = [1, 2, 3, 4];
```

→ once you assigned an value to the variable you cannot change its value through the code

Const a=10;

a=9; you cannot change like this.

→ nested arrays

Nested Arrays → multi dimensional

array of arrays

```
> let nums = [ [2, 4], [3, 6], [4, 8] ];  
< undefined  
  
> nums  
< ▼ (3) [Array(2), Array(2), Array(2)] ⓘ  
  ▶ 0: (2) [2, 4]  
  ▶ 1: (2) [3, 6]  
  ▶ 2: (2) [4, 8]  
    length: 3  
  ▶ [[Prototype]]: Array(0)
```

```
> let nums = [ [1, 2], [3, 4], [4, 5] ];  
< undefined  
  
> let nums = [ [2, 4], [3, 6], [4, 8] ];  
< undefined  
  
> nums  
< ▶ (3) [Array(2), Array(2), Array(2)]  
  
> nums.length  
< 3  
  
> nums[0]  
< ▶ (2) [2, 4]  
  nums[0].length  
< 2  
  
> nums[0][0]  
< 2  
  
>   
<   
  I
```

Nested Arrays

```
> let nums = [ [2, 4], [3, 6], [4, 8] ];
```

	0,0	0,1	
	2	4	
1,0	3	6	1,1
2,0	4	8	2,1

nums[0][0]
nums[0][1]
nums[0][1]

nums[0][0] // 2